

Help & Reference v1.2.0

Covers app v0.1.0 · Updated 2026-05-19 · Running app v0.1.0

1. About AI Switchboard

1.1 What it is, what it isn't

AI Switchboard helps a human make better decisions by turning multiple AI models into a governed deliberation council — with preserved dissent, human authority, and auditable decision memory.

A personal AI decision board for builders, writers, researchers, and technical creators who need more than an answer. They need the reasoning trail.

AI Switchboard (a.k.a. the AI Conclave) is a local background service that lets AI coding agents — Codex, Gemini, and Claude Code — consult one another through structured deliberation instead of free-form chat. You ask a question; the conclave deliberates; you get a structured verdict with disagreements surfaced verbatim, not flattened.

It offers three deliberation modes:

- **conclave** — N equal participants, full-mesh visibility, no primary. Use for genuine multi-AI deliberation.
- **resolve** — open-ended, primary-driven loop that terminates when the primary signals it's done. Use to drill to an answer.
- **consult** — fixed three-step exchange for a quick second opinion.

What it deliberately is *not*: it is not an autonomous code-execution system. The conclave is the **deliberation layer**. The interactive CLI you're already driving — Claude Code, Codex, or Gemini — is the **execution layer**. Switchboard talks, records, surfaces disagreement, and keeps the audit trail. Your CLI session does the actual editing, running, and iterating. Trying to merge those two roles was considered and explicitly declined; see

[docs/ROADMAP.md](#).

1.2 Single-user, local-only

Switchboard runs as a FastAPI service bound to **127.0.0.1:8787**. It is single-user and local-only. There is no authentication layer because there is no remote surface — the service does not bind to a public interface, and nothing in the design assumes more than one user.

State lives in a SQLite database called **switchboard.db** on your machine, with WAL mode and a 30-second busy timeout for concurrency hardening. Per-task sandboxes, uploads, exports, and logs share the same runtime-state root. Running from a repo checkout, that root is

<repo>/data/; a packaged install resolves to the platform-conventional user-data directory (e.g. **%LOCALAPPDATA%\AI Switchboard** on Windows). See *14.4 Where state lives* for the full table. None of this is committed to git, and none of it leaves your machine unless you explicitly export it.

What this does and doesn't mean for privacy. The orchestration plane — the service, the dashboard, transcripts, decision records, and API keys — never leaves your machine. The deliberation content itself does: every prompt sent to a CLI seat traverses that CLI's provider (OpenAI for Codex, Anthropic for Claude Code, Google for Gemini), and every prompt to an open-weight seat traverses OpenRouter (and then whichever provider hosts the model). Switchboard does not *add* a third party — it uses the providers you already have a relationship with — but it does not *remove* one either. The meaningful claim here is **control of the audit trail and orchestration state**, not privacy of the prompt content.

One bounded privacy lever does exist: OpenRouter requests go out with **provider.data_collection: "deny"** by default in **config.example.yaml**, which keeps prompts off the OpenRouter providers that retain or train on them. This applies only to the open-weight seats; the CLI seats are governed by whatever terms you have with OpenAI / Anthropic / Google.

1.3 Provenance and audit philosophy

Every task carries a **source_agent** field recording which CLI submitted it — **claude-code**, **codex**, **gemini**, **dashboard**, or **api**. Every slash command passes **--invoked-by <tool>** so provenance is recorded accurately, not guessed.

Inside a task, every prompt and every response is written to **agent_messages**, with per-call metadata in **agent_runs** (token counts, USD-equivalent cost where the provider reports it, timing, errors). Every message names its sender, its role, and its message type. Nothing is anonymized; nothing is summarized away.

Decisions you record via `/decide` are **permanent**. They become the binding artifact on the task, surface in the dashboard's Decision panel, and get auto-injected into any threaded follow-up via `/continue` so the next conclave sees what you actually chose last time.

The point of all this: reasoning is *inspectable* (you can read the full transcript), *revisable* (you can `/continue` a thread with new context), and *reusable* (decision records in `docs/decisions/` become context for future work).

2. Quick Start

2.1 First task (fake-adapter smoke test)

Before you spend any real-CLI quota, run a smoke test against the bundled `fake` adapter. It is an API-only adapter (hidden from the dashboard's agent list by default) that returns a deterministic response without calling out to any model — it exists specifically to confirm the orchestrator, worker, and database wiring all work end-to-end.

With the service running on `127.0.0.1:8787`, open a PowerShell terminal in the project root and submit the bundled fake-adapter task:

```
curl.exe -X POST http://127.0.0.1:8787/api/tasks `
-H "Content-Type: application/json" `
--data "@examples/task_request_fake.json"
```

You will get back a JSON body of the shape `{"task_id": "tsk...", "status": "pending", ...}`. Within about 2 seconds the background worker claims the task; within another 1 to 2 seconds the fake adapter completes the `consult`-mode exchange defined in `examples/task_request_fake.json`.

Open the dashboard's `Inbox` tab. The new task appears at the top in `completed` status. Click into it and you will see a populated transcript and a final result panel. If that worked, the orchestrator is healthy and you can move on to a real conclave with confidence.

2.2 First real conclave

Once at least one real CLI adapter (Codex, Gemini, or Claude Code) reports as available, you can submit a real conclave from the dashboard.

Open `http://127.0.0.1:8787/` in your browser and click the `New Task` tab in the top bar. In the form:

1. Set `Mode` to `conclave`.
2. Under `Agents`, check the seats you want at the table. The default 3-AI deliberation is `codex`, `gemini`, and `claude-code`. Only agents the orchestrator can currently reach show up in this list.
3. Leave `Permissions` on the `read-only` preset unless you have a reason to widen it.
4. Optionally fill in `Project path` and tick `Provide a read-only sandbox copy` if the agents should be able to browse your source while deliberating.
5. Type your question in the `Question` textarea.
6. Click `Submit task`.

The form redirects you into the `Detail` view for the new task. While the conclave is in flight, the `live activity` banner at the top of the detail panel shows which agent is currently being called and how long the current call has been running. The transcript fills in below as each agent responds. When the conclave converges (or a backstop fires), a `Final result` panel appears.

2.3 Recording your first decision

The conclave's job ends at a recommendation. The decision is still yours — and recording it is what turns a transient deliberation into a permanent piece of the project record.

After the task completes, scroll past the `Final result` panel in the `Detail` view. You will see a `Decision` panel. Type your decision in your own words — what you are choosing, and (optionally) why, what you are rejecting, known risks, and open questions. The Conclave Charter suggests this fuller shape, but a single-line decision is acceptable too.

Click `Save`. Your text is now the task's permanent decision record. From this point on:

- The decision is visible in the `Detail` view above the post-task action bar.
- The decision is included as auto-context in any threaded follow-up you start with `/continue` or the dashboard's `Continue this thread` button — every participant in the follow-up conclave sees your prior call.
- `/decision <task>` fetches it back later along with the original question and final answer, ready to act on.
- If you click `Export to decision record`, the decision is written into the exported markdown alongside the transcript and final result.

Recording a decision does not lock anything — you can edit and save again at any time.

3. The Three Modes

3.1 conclave

`conclave` mode runs N equal participants with full-mesh visibility. There is no primary and no consultant — every participant sees every other participant's previous turn, every round.

Each round, each participant posts one `ConclaveTurn` containing their current `position` plus a `convergence` signal: `i_am_done`, `still_thinking`, or `need_user_input`. The mode terminates when at least `convergence_threshold` of the participants signal `i_am_done` in the same round. The default threshold is `1.0` — unanimous agreement.

If convergence is weak (some say done, others still thinking), the orchestrator runs **one focused synthesis round**, then hands the result to a **convergence judge** — one participant arbitrates whether the remaining differences are semantically equivalent, and the orchestrator upgrades `agreement_level` accordingly. The orchestrator never picks a winner on substance; if real disagreement persists, every position is surfaced to you.

Use `conclave` for genuine multi-AI deliberation: architecture choices, contentious refactors, library selections, security calls — anything where you want three independent perspectives reasoning about the same stable situation, with disagreements preserved rather than averaged away.

3.2 resolve

`resolve` mode is open-ended and primary-driven. One agent is the `primary`; others (optional) are consultants who critique each round. Each turn the primary picks one of four `resolution_status` values:

- `resolved` — primary believes the task is done. Consultants get one more round to push back via `wants_continuation`.
- `needs_more_rounds` — primary explicitly wants another iteration.
- `needs_user_input` — primary cannot proceed without info from you. The task pauses at `awaiting_user_input` until you answer; then it resumes.
- `cannot_resolve` — primary determined the task cannot be solved with available tools, permissions, or info. The loop terminates immediately.

Termination is goal-based, not turn-capped — but there are three backstops to bound cost: `max_seconds` (total task wall time), `max_rounds` (round count), and a **repetition guard** that stops the loop if agents start repeating prior content.

Use `resolve` when you want one agent to drive ("let Codex handle this") and the others to act as checks per round. It's the right mode for debugging, deep dives, and tasks where you expect iteration toward a real answer rather than a vote.

3.3 consult

`consult` mode is a bounded three-step exchange: the **primary proposes**, the **consultant(s) critique**, the **primary finalizes**. That's it. After the primary's `primary_final` message, the task terminates.

Unlike `resolve`, the primary's `resolution_status` is ignored — the shape is fixed at three steps regardless of how confident anyone is. Unlike `conclave`, the consultants do not get equal standing; they critique once and the primary owns the final answer.

Use `consult` when you want a *quick* second (or third) opinion rather than a full deliberation. Code review of a small change, a sanity check on a design sketch, a "did I miss anything obvious" pass — anything where one or two critique rounds is enough and you don't want to spend the time or quota on a full convergence loop.

From the CLI: `/consult <agent> <question>` picks one consultant; `/secondopinion` targets the current conversation.

3.4 Picking the right mode

Pick the mode by what kind of answer you want:

You want...	Use
Three independent perspectives, with disagreements preserved verbatim and a real convergence signal	<code>conclave</code>
One agent driving to a concrete answer, with others acting as checks, iterating until done	<code>resolve</code>
A fast sanity check or second opinion on something you've mostly figured out	<code>consult</code>

Heuristics that work in practice:

- **Architecture / library / refactor debates** where multiple paths are defensible — `conclave`. You want the disagreements.
- **Debugging a real failure** or chasing down a root cause — `resolve`. You want iteration.
- **"Does this PR design look right?"** — `consult`. You want one critique pass, not a debate.
- **"Did I miss a risk in this decision I already mostly made?"** — `consult` or `/secondopinion`.

If you're unsure, default to `conclave` for design questions and `resolve` for execution-style questions. You can always re-submit in a different mode; the threading machinery (`/continue`) keeps prior context intact.

4. The Council

4.1 CLI seats

Three of the four built-in council seats are **CLI seats** — local-machine command-line tools the adapter shells out to. They are:

- **Codex** — invoked as `codex exec --json`.
- **Gemini** — invoked as `gemini -p -o json`.
- **Claude Code** — invoked as `claude -p --output-format json`.

All three are **read-only by default**. The adapter never grants write or execute permissions during a deliberation — that's a Charter-level decision, not a configuration toggle (see the ROADMAP's "Layer 2" section for why in-conclave execution was intentionally not built). The CLIs read files when the task has a sandbox, but they cannot patch them.

All three are also **JSON-output-disciplined**: every adapter asks the CLI for structured JSON and refuses to fabricate structure if it gets prose back. If a CLI returns something unparseable, the orchestrator records an `agent_error` rather than guessing.

Because these are CLIs and not API calls, each seat defaults to your **existing subscription** on that tool — your ChatGPT Plus / Pro plan for Codex, your Gemini Advanced plan for Gemini, your Claude Pro / Max plan for Claude Code. Switchboard doesn't bill anything; the CLI you already pay for handles auth and quota. The adapter just speaks to the binary on `$PATH`.

4.2 Open-weight API seats

Alongside the three CLI seats, you can mount one or more **open-weight API seats** — non-frontier (mostly) open-weight models reached over HTTP. These bring a voice from outside the OpenAI / Google / Anthropic axis into the deliberation: different training data, different priors, different blind spots.

The default backing is **OpenRouter**, a unified gateway to many providers (DeepSeek, Qwen, Z.ai/GLM, and more). The shipped `config.example.yaml` registers three seats out of the box — `deepseek`, `glm`, and `qwen` — and you can add or rename any of them. OpenRouter is **pay-per-token, no subscription**; a single conclave turn costs roughly **\$0.001–\$0.02**. Auth is via `OPENROUTER_API_KEY` (env var wins) or the dashboard's Settings → API Keys panel.

By default, OpenRouter requests are sent with `provider.data_collection: "deny"`, which keeps prompts off providers that retain or train on them — appropriate when the conclave is reviewing your code. Flip `data_collection: allow` in the `openrouter:` config section if you want to opt back in.

4.3 Adding or disabling a seat

Seats are **config-driven**. To add one, edit `config.yaml` under `openrouter.models[]` and append an entry. The shape is:

```
openrouter:
  enabled: true
  data_collection: deny
  models:
    - name: deepseek
      model_slug: "deepseek/deepseek-chat"
      max_context_chars: 400000
    - name: glm
      model_slug: "z-ai/glm-4.6"
      max_context_chars: 400000
```

The three fields are:

- `name` — the friendly council label that appears on the New Task checkbox list.
- `model_slug` — the provider's model identifier on OpenRouter.
- `max_context_chars` — your declared prompt-size ceiling for that seat. The adapter uses it to budget the inlined sandbox. Set it conservatively below the model's true token window; the OpenRouter adapter also *learns* a tighter ceiling automatically if the provider rejects an over-long request.

Restart the service after editing — the registry reads `config.yaml` at startup. The new seat then appears as a checkbox on the New Task form alongside `codex`, `gemini`, and `claude-code`.

To **disable** a seat, either remove its entry from `models[]`, or set the whole adapter's `enabled: false` (e.g. `openrouter.enabled: false`). A disabled adapter stays registered but reports `agent_unavailable`.

4.4 Per-task seat selection

You do not have to use every available seat on every task. The **New Task form** shows every registered, available agent as a checkbox — the three CLI seats plus any OpenRouter seats you've configured. You tick the boxes for the seats you want to sit on this particular conclave.

A **two-agent conclave is valid**. So is a three-agent or five-agent one. The convergence machinery doesn't assume a fixed N — it works off the threshold (default unanimous) over whichever participants you selected. A consult-mode task uses one primary plus one or more consultants; a resolve-mode task uses one primary; a conclave-mode task uses everyone you ticked, equally.

Reasons you might trim the roster:

- Cost.** The CLI seats use your existing subscriptions; the API seats meter tokens. Leaving the API seats off keeps a task strictly free.
- Speed.** Fewer participants means fewer parallel calls per round and a shorter critical path.
- Topical fit.** If a model is weak on your domain, leaving it off the panel is cleaner than discounting its vote.
- Availability.** A seat with a missing API key or an unavailable CLI shows up disabled; you simply don't tick it.

The task's recorded transcript and decision reflect only the seats you selected — the deliberation is shaped by who's actually in the room.

4.5 Model selection philosophy

This system prioritizes **top-shelf model quality over per-token cost**. The conclave's value depends on each agent reasoning genuinely well about your question — saving a few cents per turn by picking a weaker model defeats the purpose. The shipped defaults are the top-tier model in each provider's lineup at the time of writing.

Provider (seat)	Declared model	\$/M in · out	Why this one
OpenAI (<code>codex</code>)	<code>openai/gpt-5.5</code>	\$5 · \$30	Flagship general model. <code>gpt-5.5-pro</code> is ~6× the cost for marginal gains; not worth it.
Anthropic (<code>claude-code</code>)	<code>anthropic/claude-opus-4.7</code>	\$5 · \$25	Top reasoning model. The <code>-fast</code> variant is ~6× the cost for marginal speed.

Provider (seat)	Declared model	\$/M in · out	Why this one
Google (<code>gemini</code>)	<code>google/gemini-2.5-pro</code>	\$1.25 · \$10	Current stable flagship. <code>3.1-pro-preview</code> is newer but preview-tagged.
OpenRouter open-weight	<code>deepseek/deepseek-chat</code> , <code>z-ai/glm-4.6</code> , <code>qwen/qwen3-coder</code> , <code>moonshotai/kimi-k2.6</code>	\$0.22–\$0.73 · \$0.89–\$3.49	Outside-axis voices — different training data, different blind spots than the OpenAI/Anthropic/Google trio.

To change any of these: edit `agents.<name>.model_slug` in `config.yaml` for a CLI seat, or edit/append `openrouter.models[]` for an open-weight seat. Restart the service to pick up the change.

4.6 CLI auth modes (subscription vs API)

Each frontier CLI can authenticate two ways:

- **Subscription / OAuth (default).** You log in via the CLI itself — `claude /login` , `codex login` , `gemini /auth` . No per-token charges; your provider subscription absorbs the cost (Claude Pro/Max, ChatGPT Plus/Pro, Gemini Advanced).
- **API key.** You log in via API key inside the CLI's auth flow, or set the corresponding env var. The CLI then bills per token at the provider's published rates. Significantly more expensive than subscription on heavy use, but useful if you don't have a subscription or want explicit per-token visibility.

You can switch between the two at any time from inside the CLI's own auth menu (it tends to be something like `/login` , `/auth` , or `logout` + re-login with the alternative method). The Pricing view detects which mode each CLI is currently in and labels its row accordingly — see 4.8 How model & auth detection works for the technical detail of how this is detected.

Important: CLI changes require a Switchboard service restart to be reflected in the Pricing view. The auth-state files are re-read on every request, but the Python process holds its own image of the code path; the restart isn't for the auth detection itself, it's a general fact of how uvicorn runs without `--reload` .

4.7 The Pricing view

Click the violet `$` glyph in the sidebar (between the help `?` and the gear) to open a sortable pricing table. Columns:

- **Agent** — the council seat name as configured.
- **Type** — a colored pill: `cli` (CLI in subscription mode, green), `cli-api` (CLI in API mode, amber), `openrouter` (open-weight pay-per-token, violet).
- **Model in use** — the detected slug from the CLI's own config when available, otherwise the slug declared in `config.yaml` . A small "detected · <source>" caption shows where the value came from. If declared and detected disagree, a yellow *drift* chip appears with hover text explaining how to resolve it.
- **Context** — the model's context window from OpenRouter's catalog.
- **Input \$/M, Output \$/M** — per-million-token rates pulled live from OpenRouter (which carries Anthropic, OpenAI, Google, and open-weight pricing in one unified catalog). Cached server-side for 5 minutes so view-open is near-instant after first load.
- **Est. per turn** — input × 5,000 tokens + output × 1,000 tokens (typical conclave turn). Click the column header to sort; the default sort is most expensive first.
- **Notes** — explains the detected source for transparency: e.g. "API mode (~/.codex/auth.json (auth_mode=apikey))" or "Subscription mode (~/.gemini/settings.json (selectedType=oauth-personal)) — no per-token charges."

Subscription-mode rows leave the cost columns blank because there is no per-token rate — your subscription pays.

4.8 How model & auth detection works

Detection runs server-side on every request to `/api/agents/pricing` . The OpenRouter pricing catalog is cached for 5 minutes; the CLI auth-state files are read fresh on every request.

Auth mode resolution order:

1. The CLI's own auth-state file (most authoritative — reflects what you actually selected via the CLI's login flow).
2. Env var presence (fallback signal of API intent if the file gives no clear answer).
3. Default to subscription.

Per-CLI signals:

CLI	File	API-mode signal	Subscription-mode signal
Codex	<code>~/.codex/auth.json</code>	<code>auth_mode: "apikey"</code>	<code>auth_mode: "chatgpt"</code> (or any non-apikey value)
Gemini	<code>~/.gemini/settings.json</code>	<code>security.auth.selectedType</code> contains "api" or "key" (e.g. <code>"gemini-api-key"</code>)	<code>selectedType: "oauth-personal"</code> (or any other non-API value). Falls back to <code>~/.gemini/oauth_creds.json</code> existence if <code>selectedType</code> is missing.
Claude Code	<code>~/.claude/.credentials.json</code>	(no explicit marker known yet — falls through to env var)	File presence with non-zero size

Model identity resolution:

CLI	Where the model lives	What's read
Codex	<code>~/.codex/config.toml</code>	<code>model = "..."</code> at the top level. Prepended with <code>openai/</code> to form an OpenRouter slug.
Claude Code	<code>~/.claude/settings.json</code>	<code>model</code> field if present (written by the <code>/model <name></code> command inside Claude Code). Aliases like <code>opus</code> / <code>sonnet</code> / <code>haiku</code> are mapped to current OpenRouter slugs.
Gemini	(no persistent model preference detected)	Falls back to the slug declared in <code>config.yaml</code> .

When the detected model differs from the declared one, the Pricing view shows the **detected** value (because it reflects what your CLI is actually using) and flags the mismatch with a "drift" chip. To resolve drift, either change the CLI's model (e.g. `/model opus` inside Claude Code) to match the declared slug, or edit `agents.<name>.model_slug` in `config.yaml` to match the detected one.

Caveats:

- CLI auth file formats can change between CLI versions; if a detection stops working, check the file's current shape and update `_detect_cli_auth_from_files` in `app/api/agents.py` accordingly.
- The env-var fallback only applies if the file gives no clear signal — it does *not* override an explicit file-based selection.
- Switchboard's CLI adapters do not pass `--bare` or equivalent flags that would force a specific auth mode; the adapter trusts the CLI's own current selection.

5. The Charter

5.1 Operability before capability

Charter v1.2 codifies a principle the conclave converged on after deliberating its own evolution: **operability before capability**. When the conclave evaluates a proposal to add a new feature, mode, agent type, permission layer, or piece of heavy infrastructure, it must first weigh the impact on the trust foundations of the existing system — *observability, durability, recoverability, audit trail, retention, and export*.

A *material tension* exists when a proposed capability would degrade those foundations, conflict architecturally with them, raise state/permission/coordination complexity beyond what they can support, or crowd out a named operability gap. In material tension, unresolved conflict resolves in favor of operability unless you (Glen) explicitly approve a bounded exception.

This isn't only a deliberation rule — it has a paper trail requirement. Every significant Switchboard capability or infrastructure decision record must include an **Operability Impact** field covering effects on those foundations, accepted risks, mitigations, any approved exceptions, and follow-up review points. See `docs/CONCLAVE_CHARTER.md` for the binding text.

5.2 Multimodal disagreement

When the conclave is reasoning about an image and participants describe what they literally see *differently* — not differing opinions but differing perception of pixels, of text, of layout — the system **does not run a synthesis round and does not force consensus**. Visual perception is a matter of fact, not deliberation; one model updating its description after seeing another's is conformity, not better seeing, and produces a less reliable answer than transparent disagreement.

Instead, the participant that detects the divergence (or the final-round participants collectively) sets `convergence: need_user_input` and emits a structured `user_input_question` containing:

- **Consensus observations** — visual elements every participant reported.
- **Disputed observations** — what each participant reported about specific elements, attributed by name.
- **Type of claim** for each — literal observation, OCR/text reading, or interpretation.
- **Confidence**, when supplied.
- **Suspected cause of divergence** and a concrete question for you.

Your answer becomes the authoritative resolution for that image. A participant whose perception was wrong does not have its non-perceptual contributions discarded — the final agreement level reflects that the disagreement was perceptual, not analytical. Visual perception disputes are first-class.

5.3 The keeper role

You are **the keeper**. The Conclave Charter (`docs/CONCLAVE_CHARTER.md`) names you as the final authority: you provide intent, context, judgment, validation, and approval. The conclave provides alternatives, critique, synthesis, and reusable decision records.

The keeper's primary act on a completed task is `/decide`. You record one of three outcomes:

- **Ratify** — accept the conclave's converged answer as your decision.
- **Reject** — decline the conclave's answer; your decision text states what you're doing instead.
- **Return to conclave** — kick the question back via `/continue` with new framing or evidence.

The keeper **does not edit conclave output**. Once the conclave has converged, the transcript and final answer are immutable artifacts. Your decision is a separate, authoritative record placed alongside them — not a rewrite of what the agents said. This separation is what keeps the audit trail clean: the conclave's reasoning is preserved verbatim, and your binding call is preserved verbatim, and the two are distinguishable forever.

Note: within the charter itself, the `claude-code` adapter is the keeper of the *charter document* — meaning it maintains the canonical file, tracks amendments, and restates working rules when participants drift. That is a continuity role, not authority. Authority is yours.

5.4 Amendment process

The Conclave Charter is binding and embedded in every prompt to every participant in every mode. Changes are not edited in place by anyone acting alone — they go through a formal amendment process defined in the charter itself.

The process:

1. Any participant, or you, proposes an amendment.
2. The amendment is debated in a `conclave`-mode task on Switchboard. The three AIs deliberate on the proposed wording.
3. You ratify or reject the outcome via `/decide`.
4. On ratification, the keeper bumps the version number and updates the canonical file at `skills/generic/conclave_charter.md` (with the human-readable mirror at `docs/CONCLAVE_CHARTER.md`).
5. A numbered decision record is filed at `docs/decisions/<NNNN>_<slug>.md`.

This is how the charter actually got from v1.0 to v1.2: the Multimodal Disagreement section (v1.1) and the Operability-before-capability principle (v1.2) were each proposed in conclave deliberations, ratified, versioned, and recorded as decisions 0002 and 0006. The charter is a living document; conflicts between it and updated practice should be raised and resolved through amendment, not by quietly editing files.

6. Using the Dashboard

6.1 Inbox

The **Inbox** tab is the primary list view for every task the orchestrator knows about. It auto-refreshes every 5 seconds and uses a sticky table header so the column labels stay visible as you scroll through long lists.

The filter bar above the table lets you narrow the view:

Filter	What it does
Status	Limit to pending , running , awaiting_user_input , completed , failed , or cancelled .
Mode	Limit to conclave , consult , or resolve .
Export status	Exported shows only tasks already written to the exports directory; Not exported shows only terminal tasks that have not been exported yet.
Search	Case-insensitive substring match across task ID, the user request, your recorded decision, and the conclave's final answer. Runs whole-DB server-side — independent of the Show last page size — and returns the newest matches.
Show last	How many rows to fetch: 50, 100, 500, 1000, or 5000.

The **X** button beside the search box resets all filters to defaults. A counter under the filter bar shows how many tasks are visible out of the total fetched.

The **Bulk export unexported** button writes every unexported terminal task (**completed**, **failed**, or **cancelled**) to the exports directory as a markdown decision record. Tasks that succeed are marked exported so the next bulk run skips them.

6.2 New Task form

The **New Task** tab is where you compose and submit a new deliberation. Every field maps directly onto the task request the orchestrator stores in SQLite.

- Mode** — one of **conclave** (N equal participants, deliberate to convergence), **consult** (primary proposes, consultant critiques, primary finalizes), or **resolve** (primary-driven loop with backstops). The hint line under the dropdown summarizes the selected mode.
- Agents** — a checkbox list of every registered, user-facing agent. Each row carries a small coloured dot pulled from **/api/health**: green if the seat is ready, red if unreachable (with the remediation hint as a tooltip), grey if readiness hasn't resolved yet. You can still tick a red seat — local knowledge may be ahead of the 30-second readiness cache — but the form will ask for confirmation at submit time. See 6.6 *Seat readiness signals* for the full state machine.
- Permissions** — collapsed by default. Inside, three preset buttons quickly set the checkboxes: **Read-only**, **Read + .env**, and **Read everything**. You can also tick individual permissions by hand.
- Project path** — optional. When set, the orchestrator scopes permissions to this directory. Tick **Provide a read-only sandbox copy** to give each agent a per-task read-only snapshot of the project.
- Question** — the prompt itself. The **Attach current git diff** button beside the label is enabled once **Project path** is set.
- Attachments** — drag files onto the dropzone, or click **browse**. Text, Markdown, PDF, code, and image files are supported up to 20 MiB each.

Click **Submit task**. You are redirected to the **Detail** view for the new task.

6.3 Detail view

The **Detail** view is where you watch a task run, read the transcript, record a decision, and act on the result. From top to bottom:

- Thread breadcrumb** — appears only if the task is part of a thread; shows the ancestry chain so you can jump back to any prior node.
- Live activity** — visible while the task is running. Shows the currently-active agent and elapsed time on the current call, so you always know whether the conclave is making progress or stuck.
- User request** — the question exactly as submitted.
- Answer prompt** — appears only when the task pauses with **status: awaiting_user_input**. Shows the council's question, optional context for why they are asking, and a textarea to send your reply.
- Transcript** — every message in the deliberation, including each agent's position and convergence signal per round.
- Final result** — the structured outcome once the task terminates.

- **Decision panel** — your authoritative decision in your own words; save and edit at any time.
- **Post-task action bar** — `Start New Task`, `Continue this thread`, `Export to decision record` (writes markdown to the exports directory under the runtime state root), and `Download...` for the full detail as PDF, DOCX, MD, or TXT.

6.4 Settings

The `Settings` view is where you store the API key for the pluggable open-weight council seats. Open the left rail, click the gear icon, and choose `Settings`. The `API Keys` panel has one row, for OpenRouter.

The row has this shape:

- A password input (the key is masked by default).
- An eyeball button that toggles the input between password and plain text so you can verify what you pasted.
- A `Save` button (writes the key to the local `switchboard.db` SQLite database under the runtime state root).
- A `Clear stored key` button (removes it from the database).

Precedence rule. The environment variable always wins over the stored value. If `OPENROUTER_API_KEY` is set in the shell you launched the service from, that key is used for OpenRouter calls regardless of what is saved in the database. The database entry is a fallback used only when the env var is absent.

OpenRouter backs the open-weight seats (`deepseek`, `glm`, `qwen`, `kimi`); a conclave turn typically costs \$0.001–\$0.02 pay-per-token.

6.5 One-click desktop launcher

Once the manual `uvicorn` invocation works, there's a double-clickable launcher so you don't have to keep a terminal open. The launcher (`launch.pyw` at the repo root) polls `/api/health` first: if the service is already up, it just opens a fresh browser tab. If not, it spawns `python -m uvicorn app.main:app` as a fully detached background process, waits up to 30 seconds for `/api/health` to respond, then opens the dashboard. Closing the launcher does not kill the service.

Windows. Run `tools\install-desktop-shortcut.ps1` in PowerShell. It creates `AI Switchboard.lnk` on your Desktop, pointing `pythonw.exe` at `launch.pyw` with the AI Conclave logo as the icon. Right-click → *Pin to taskbar* for one-click access from the taskbar.

macOS. Run `tools/install-desktop-app.sh`. It builds an `AI Switchboard.app` bundle on your Desktop; drag it onto the Dock to pin. First launch may trip macOS's "cannot check for malicious software" warning — right-click → *Open* → *Open* clears it permanently.

Linux. No bundled installer; drop a `.desktop` file in `~/.local/share/applications/` pointing at `python3 /path/to/launch.pyw`.

Launcher events plus uvicorn's stdout/stderr land in `<repo>/data/launcher.log` — that's the place to look when a browser tab fails to appear. To stop the service: kill the python process (Task Manager / `pkill -f 'uvicorn app.main'`) or delete the `switchboard.pid` file at the runtime state root.

6.6 Seat readiness signals

Per DR0017, `/api/health` returns a `seats: [...]` array alongside the service status — one entry per registered, user-facing adapter, with `available`, a short machine-stable `reason`, and a user-facing `hint`. The dashboard polls this and paints the dot next to each agent checkbox on the New Task form. Reasons you'll see:

reason	Meaning
ok	Seat is reachable. CLI binary on PATH (or at <code>command_path</code>); OpenRouter API key present.
command_not_found	CLI adapter could not locate the binary. Install the CLI or set <code>agents.<name>.command_path</code> in <code>config.yaml</code> to an absolute path.
configured_path_missing	<code>command_path</code> is set but no file exists at that path. Strict precedence — the adapter will not silently fall back to PATH. Fix the path or unset it.

reason	Meaning
api_key_missing	OpenRouter seat without OPENROUTER_API_KEY in the env or a stored key in Settings.
readiness_check_failed	The adapter raised while computing readiness. Detail is in the hint; check the service log.

The seats array is cached server-side for 30 seconds so chatty dashboard polling doesn't spawn a `shutil.which` + HTTP probe per call. After fixing a missing CLI or pasting a fresh API key into Settings, expect up to a 30-second lag before the dot turns green; a service restart clears the cache immediately.

The `command_path` escape hatch. Windows double-click and macOS Finder launches don't inherit your shell PATH. If a CLI works in your terminal but the adapter reports `command_not_found`, the cause is usually GUI PATH inheritance — paste the absolute binary path into `agents.<name>.command_path` in `config.yaml` and restart. This is documented inline in `config.example.yaml` next to each CLI agent's `command` field.

7. Slash Commands

7.1 /conclave, /consult, /secondopinion

You can submit tasks to Switchboard without leaving the CLI session you are already coding in. From your Claude Code, Codex, or Gemini session, the same three commands cover the most common cases.

- `/conclave <question>` — runs a 3-AI conclave deliberation with `codex`, `gemini`, and `claude-code`. This is the full deliberation: every agent posts one position per round, signals convergence, and the loop terminates when participants agree (or a synthesis round plus judge pass resolves a weak convergence). Use it for design questions, architectural calls, and anything where you want disagreements surfaced verbatim instead of flattened.
- `/consult <agent> <question>` — a quick second opinion from one named agent. The CLI you ran the command from acts as the primary (drafts the answer); the named agent acts as the consultant (critiques it); the primary finalizes. Three-step exchange, no convergence loop. Use it for review, sanity checks, and "is this refactor safe" questions.
- `/secondopinion [topic]` — runs a second opinion on the current conversation context. If you omit `[topic]`, the command uses the last substantive question or proposal in your session as the topic. Defaults to `claude-code` as primary (which has your full conversation context) and `codex` as consultant, but you can name a different consultant in your phrasing.

Codex does not have literal `/` slash commands — it activates the equivalent skill via trigger phrases ("ask the conclave", "get a second opinion"). Claude Code and Gemini get literal slash commands.

7.2 /decide, /decision

These two commands let you record and retrieve your authoritative decision on a task from inside the CLI session, without opening the dashboard.

- `/decide <task|latest> <text>` — records your final decision on a completed task. The first argument is the task ID (typically `tsk_...`) or the literal word `latest` to target the most recent task. Everything after it is the decision body — free-form text, in your own words. The Conclave Charter suggests structuring it (what was chosen, why, what was rejected, known risks, open questions, who keeps continuity), but a one-liner is accepted.
- `/decision <task|latest>` — fetches a recorded decision back along with the original question, the conclave's final answer, and the conclave's agreement level. The CLI typically follows the fetch by asking you what you want to do next; if the decision implies a concrete code change or follow-up, the CLI will surface the next step.

A recorded decision is the same record whether you wrote it via `/decide` or in the dashboard's `Decision` panel — they read and write the same field. Once recorded, the decision is auto-injected as context into every threaded follow-up you submit with `/continue`, and is included in markdown exports.

If no decision is recorded yet, `/decision` says so and points you back to `/decide` or the dashboard.

7.3 /continue, /thread, /answer

These three commands cover threading and the user-input pause cycle.

- `/continue <parent|latest> <question>` — creates a threaded follow-up task. The new task is linked to the parent via `parent_task_id` and inherits the parent's mode and agents (a 3-AI conclave begets another 3-AI conclave). The orchestrator auto-loads the ancestry — every prior task's question, final answer, and your recorded decision — into each participant's prompt as a "Prior Thread Context" section, so the follow-up picks up where the prior thread left off. Use it when the first resolution was ambiguous, when you want to drill into implementation after a design decision, or when a new question depends on prior context.
- `/thread <task|latest>` — shows the ancestry chain for a task, oldest-first, with the current task marked. Each ancestor's question, final answer, and recorded decision is summarized. Useful for recalling what was already settled in a long-running thread before posting a new question.
- `/answer <task|latest> <text>` — resolves a task paused with `status: awaiting_user_input` (the conclave asked you a question and stopped). The text after the task ID is sent as your reply. The Claude Code variant also supports a "run-and-send" mode: if the text after the task ID looks like a shell command (`pytest`, `git`, `npm`, etc.), the CLI runs it, captures combined stdout and stderr, and submits the output. Once submitted, the task flips back to `pending` and the worker re-claims it.

7.4 The --invoked-by flag

Every slash command and every skill invocation passes a `--invoked-by <tool>` flag to the Switchboard task submission. The orchestrator records the value on the task as `source_agent` — one of `claude-code`, `codex`, `gemini`, `dashboard`, or `api`.

You do not type this flag yourself. The Claude Code commands, Codex skill, and Gemini extension all set it automatically based on which surface you launched from. The flag exists so provenance is recorded by the surface, not *guessed* by the orchestrator after the fact.

The recorded provenance shows up in the Inbox table, on the task detail header, and in every export. It is also queryable via the API — useful if you later want to ask "what did I submit from inside Claude Code last week?" without having to guess.

8. Decisions & the Coding Loop

8.1 Deliberate → Decide → Execute → Record

Switchboard's intended workflow for using the conclave during real coding work is a four-step loop, documented canonically in `docs/CODING_WORKFLOW.md`. The principle: **the conclave deliberates, the interactive CLI executes, you decide**.

1. **Deliberate** — submit the question with `/conclave` (full deliberation) or `/consult` (quick second opinion). Attach relevant files or enable the project sandbox so agents can browse your code read-only.
2. **Decide** — read the transcript and the final answer, then record your authoritative call with `/decide latest "<your decision in your own words>"`. This becomes the task's permanent record and auto-injects into any threaded follow-up.
3. **Execute** — switch back to the interactive CLI session you're already running. Ask it to act on the decision (you can paste it in or use `/decision <task>` to load the full context inline). The CLI's native tools (Read, Edit, Bash, etc.) do the actual work, under your supervision.
4. **Record** — for significant work, close with a decision record markdown file in `docs/decisions/`. Capability or infrastructure changes additionally need an Operability Impact field per Charter v1.2.

When execution surfaces something the deliberation didn't anticipate — a test failure, a missed constraint — use `/continue latest <new question>` to thread a follow-up that inherits the parent's mode, agents, and context.

8.2 Why in-conclave execution wasn't built

An obvious-looking extension was considered: extend per-task permission flags through to the adapter CLIs so a conclave with `can_run_commands: true` could actually read, write, and execute during deliberation. That proposal — internally called **Layer 2** — was deliberated, declined, and deferred indefinitely. Five reasons, summarized from `docs/ROADMAP.md` § "Considered and Intentionally Not Built":

- **Deliberation needs a stable situation.** The conclave's value depends on three agents reasoning about the same fixed state. If any participant mutates the filesystem mid-loop, the others become reactors to fait accompli rather than independent analysts. The deliberation softens.
- **Race conditions.** Conclave participants run in parallel via `asyncio.gather`. Two agents editing the same file produces last-write-wins corruption with no audit trail of intent.
- **Output-discipline regression.** With elevated tool access, models drift away from strict JSON output toward tool-use exploration. Already observed once with Claude's `--permission-mode plan`.
- **Redundancy with the interactive CLI.** The Claude Code (or Codex, or Gemini) session driving Switchboard is already an execution-capable agent. Adding two more parallel writers gives you a coordination problem, not 3× capability.

- **Audit trail divergence.** Today the trail is clean: deliberate → decide → act. In-conclave execution scrambles that into arbitrary parallel writes during deliberation, which is hard to reconstruct after the fact.

The replacement is the four-step loop in `docs/CODING_WORKFLOW.md`. The interactive CLI is the execution layer; that's what it was built for. Reconsider this only if a real use case emerges that the four-step loop genuinely cannot handle.

8.3 Decision record fields

A decision record is the artifact that closes significant work. It lives as markdown in `docs/decisions/<NNNN>_<slug>.md` and is indexed in `docs/decisions/INDEX.md`. Each record covers, at minimum:

- **What was chosen.** The decision itself, plainly stated.
- **Why.** The load-bearing reasons — not exhaustive, but enough that future-you (or a future conclave) can tell why this path won.
- **What was rejected** and why. The alternatives considered and the concrete reasons they lost. This is what prevents the same proposal from cycling back without new information.
- **Known risks.** What could still go wrong with the chosen path.
- **Open questions.** What's deferred, what's unresolved, what needs evidence later.
- **Who is keeping continuity.** The named participant responsible for tracking follow-up.

For **significant Switchboard capability or infrastructure decisions**, Charter v1.2 additionally requires an **Operability Impact** field covering effects on observability, durability, recoverability, audit trail, retention, and export; complexity added; accepted risks; mitigations; any *Operability before capability* exception you approved; and follow-up review points.

Decision records are reusable context. They get cited in future conclave prompts (when threaded via `/continue`) and read by future humans (you, mostly) before proposing changes in the same area. Together with the SQLite transcript, they are the long-term memory of Switchboard's own design.

9. Attachments & Multimodal

9.1 Text, Markdown, PDF

The New Task form has an **attachments area** for adding files to the deliberation. Three text-shaped formats are inlined directly into every agent's prompt:

- **Plain text** (`.txt`) and similar
- **Markdown** (`.md`)
- **PDF** (`.pdf`) — extracted to text server-side via `pypdf`, then inlined

Each agent receives the file's content as part of its prompt — no separate "attached document" channel, no out-of-band reference the agent has to chase down. The text just appears in context, labelled by filename, so the agent can quote and cite it directly in its reasoning.

The per-file ceiling is **20 MiB**. Larger files are rejected at upload time. For PDFs, that's the raw PDF size — the extracted text is typically much smaller than the binary it came from, so the practical effective ceiling on PDF *content* is generous.

A few notes:

- PDF extraction is text-only. Images embedded inside the PDF aren't passed through to the multimodal channel; if you need agents to actually *see* a figure, save it as PNG/JPG and attach it separately (see *Images*).
- Inlined text counts against each agent's effective context window. Large attachments may force the API seats to omit some of an inlined project sandbox — the orchestrator prioritises the user-submitted attachment over auto-inlined source.
- Attached files are stored under the runtime `uploads/` directory (see *14.4 Where state lives*) and are not committed to the repo.

9.2 Images

Images are handled differently from text. The supported formats are **PNG, JPG/JPEG, GIF, and WebP**. Rather than being OCR'd or described into text, they are passed **natively** to each adapter and forwarded to the CLI through its own image-input mechanism. Each agent *sees* the actual image.

This matters because the three CLIs are multimodal-capable: Codex, Gemini, and Claude Code can all reason directly over pixels. A description-converted-to-text pipeline would lose colour, layout, spatial relationships, handwriting style, plot detail, UI element positions — exactly the things you tend to want a second opinion on. Native piping preserves all of that.

Implications you should be aware of:

- **Different agents may see different things.** Vision models are not interchangeable. A chart's axes, a screenshot's button placement, a photo's foreground subject — these can be read differently by Codex's vision stack than by Gemini's or Claude's.

- **That disagreement is preserved, not flattened.** The Charter's *Multimodal Disagreement* rule forbids the system from synthesising a consensus on a visual-perception dispute. See *When agents disagree on what they see* for what happens instead.
- **API seats (OpenRouter)** are text-only in the current adapters. If a task includes images, those seats won't receive them — only the CLI seats will. Tick that into your seat selection.

9.3 Drag-folder upload

For projects too large to attach file-by-file, the New Task form's dropzone accepts an **entire folder dragged from your file manager**. The browser walks the folder via `webkitGetAsEntry`, and the dashboard uploads each file in sequence.

Standard **ignore patterns** are applied automatically — the same set used by the project sandbox — so you don't accidentally drag in 5 GB of

`node_modules`:

- `.git`, `.hg`, `.svn`
- `node_modules`, `__pycache__`, `.venv`, `venv`
- `dist`, `build`, `target`, `out`
- `data` (so you can't accidentally re-upload Switchboard's own runtime dir)
- Common binary extensions: `.exe`, `.dll`, `.so`, `.zip`, `.mp4`, `.pyc`, etc.

Each remaining file uploads one after the other rather than in a single multipart blast — the dashboard shows progress as the queue drains. The same 20 MiB per-file ceiling still applies; oversized files are skipped with a notice rather than failing the whole drop.

The drag-folder path is the right choice for "I want the conclave to read my whole repo" when you don't have a stable `project_path` available — for instance, when you're working with code that lives on a different machine, or a scratch directory that isn't a git checkout. If you *do* have a path on disk and want a snapshot of it instead, prefer the project-sandbox toggle — it's cheaper and cleaner.

9.4 Git-diff attachment

When you're mid-edit and want the conclave to weigh in on *just the changes*, use the **"Attach current git diff"** button on the New Task form. It's enabled whenever `project_path` is set on the form.

Pressing it calls `POST /api/git/diff` on the local service. The endpoint runs `git diff` followed by `git diff --cached` server-side in the project directory, joins the two outputs, and returns the combined diff. The dashboard then **appends that diff to the question textarea** — you can edit it, add framing context above it, or trim sections out before submitting. Nothing is uploaded to a separate channel; the diff is part of your question.

This is useful when:

- You want a code review pass on what you've changed, not the whole project.
- You want to ask "is this refactor safe?" against the actual hunks, not a description of them.
- The full sandbox would blow the API seats' context budget but the diff alone fits comfortably.

You can combine the diff with a project sandbox: the diff goes into the question, the sandbox provides the surrounding code each agent reads for context. The CLI seats browse the sandbox natively; the API seats see the inlined slice. Because the diff is in your question text, it counts against the prompt budget like any other text attachment — large diffs may displace inlined source on the API seats.

9.5 When agents disagree on what they see

The Conclave Charter v1.2 contains a **Multimodal Disagreement** clause that governs what happens when agents look at the same image and report different things. The clause forbids the system from *synthesising* a visual-perception dispute into a smoothed-over consensus.

The reasoning: vision models have different perceptual stacks, and when they disagree on what an image shows, that disagreement is information. Averaging "Codex sees a red button" and "Gemini sees a green button" into "the button is somewhere on the orange spectrum" is worse than admitting the two readings conflict. The user, who has the actual eyes on the actual image, is the right adjudicator.

So instead of forcing convergence, the orchestrator **surfaces the disagreement** to you:

- Each agent's reading of the image is preserved verbatim in the transcript.
- The dissent is flagged in the final result rather than dissolved into the synthesis.
- You decide which reading is correct — typically by re-examining the image yourself, possibly recording a `/decide` that ratifies one reading and explains why.

This applies specifically to *visual-perception* disputes — disagreements about what is present in pixels. Disagreements about *interpretation* of an image whose contents the agents agree on (e.g. "what does this architecture diagram *imply*?") are normal deliberation territory and run through the usual

convergence machinery. The charter draws the line at perception, not at reasoning about perceived content.

10. Project Sandbox

10.1 The read-only copy

When you tick **"Provide a read-only sandbox copy"** on the New Task form and `project_path` is set, the orchestrator copies your project to a per-task directory under `<runtime-state-root>/sandboxes/<task_id>/` before the deliberation starts. Each agent then gets **read-only access** to that copy via its native mechanism:

- **Codex** — `-C <sandbox>` plus `-s read-only`
- **Claude Code** — `--tools "Read" --add-dir <sandbox>`
- **Gemini** — `--include-directories <sandbox>` plus `--approval-mode plan`

Standard ignore patterns are applied during the copy: `.git`, `__pycache__`, `node_modules`, `.env`, `dist`, `data`, and common binary extensions (`.exe`, `.so`, `.zip`, etc.). Files named like environment / secret material (`.env`, `*.key`, `credentials*`, `id_rsa*`) are skipped unless the task explicitly granted `can_read_env_files` or `can_read_secrets`.

Hard caps: **1 MiB per file** and **200 MiB total**. If your project exceeds the total cap, the copy stops at the limit and the agents see a partial snapshot.

The crucial property: the sandbox is a **snapshot**, fixed at task-creation time, and the agents have **no write or execute path** back into it. Your original project directory is never touched — read or written — by any participant in the deliberation. In-conclave write/execute was considered and intentionally not built; see the ROADMAP's "Layer 2" section.

10.2 Inline file tree for API seats

The CLI seats browse the sandbox natively via their own tools. The **API seats — OpenRouter — cannot**. They are HTTP calls to a chat-completions endpoint; there is no `Read` tool, no `--add-dir`, no shell. They only receive prompt text.

So when a task has a sandbox attached, the API adapters do something different: they **inline a read-only view of the project** directly into the prompt. The work happens in `app/utils/sandbox_inline.py`, and the section is built fresh for every call. It contains:

- A **file manifest** of every inlineable file with its size, so the model has the lay of the land even if not every file fits.
- The **contents** of as many files as fit in that seat's character budget.
- An **omission note** stating how many files were left out, so the model knows the view is partial and can ask for specific paths if needed.

File inclusion is **priority-ordered**: known entry-point names (`main.py`, `index.ts`, `package.json`, `pyproject.toml`, `README`, `Dockerfile`, ...) come first, then shallow source/config files, then deeper ones. Binaries, generated assets, and files larger than 200 KB are skipped.

The budget is per-seat: each model's `max_context_chars` from `config.yaml`, minus headroom for the base prompt and response. If the OpenRouter adapter learns a tighter real ceiling from a provider's "context length exceeded" reply, the inlined section is re-sized automatically on the retry. See decision 0012 for the rationale.

CLI seats get the sandbox path but not the manifest. Per DR0018, the `## File Manifest` block is only emitted to OpenRouter seats. Codex, Claude Code, and Gemini still receive the sandbox path and the read-only tool guidance in their prompt — they just don't get the static file listing, because their native tools (Codex's `-C <sandbox>` shell, Claude's `Read` / `Glob` via `--add-dir`, Gemini's `--include-directories`) produce a richer, on-demand view of the same tree. Cuts 5–30 KB out of every CLI prompt — budget reclaimed for `prior_messages` in deep continue-threads.

10.3 Cleanup and orphan sweep

Sandboxes are **cleaned up when a task terminates**. The orchestrator calls `cleanup_sandbox(task_id)` on the task's completion path, which removes `<runtime-state-root>/sandboxes/<task_id>/` from disk. The original project is, again, never touched — only the per-task copy is removed.

That covers the happy path. The unhappy path is the one where the service is killed mid-task: the worker process exits before reaching its cleanup, and a sandbox directory is left behind. Without intervention, these would accumulate across crashes and consume disk indefinitely.

The mitigation is the **orphan sweep**. On service startup, `sweep_orphan_sandboxes()` runs once. It:

- Lists every directory under the `sandboxes/` root.

- Builds the set of *active* task IDs from the database — anything in `pending`, `running`, or `awaiting_user_input` state.
- Deletes any sandbox directory whose name is **not** in that active set.

The net effect: every restart of the service self-heals the sandbox directory. A clean shutdown leaves nothing for the sweep to do; a crash leaves orphans that the next startup removes. The startup log line tells you how many were swept, if any.

If you ever want to inspect a sandbox by hand, look under `<runtime-state-root>/sandboxes/<task_id>/` while the task is still alive — once the task reaches a terminal state, that directory is gone.

11. Cost, Tokens & Retention

11.1 Per-message and aggregate cost

Every agent invocation writes an `agent_run` row that records the input and output token counts the provider returns, plus a USD-equivalent cost when the provider exposes one. The Detail view shows these per-message inline (alongside each turn in the transcript) and totals them on the post-task bar once a task reaches a terminal state, so you can see at a glance what a deliberation actually cost.

OpenRouter is the seat that reports money directly: each call is metered and lands in the roughly `$0.001` to `$0.02` per turn range for the four open-weight models bundled in `config.example.yaml` (`deepseek`, `glm`, `qwen`, `kimi`). Those numbers show up verbatim in the per-run accounting.

The CLI seats (`codex`, `gemini`, `claude-code`) default to subscription auth — your ChatGPT Plus/Pro, Gemini Advanced, or Claude Pro/Max session — so the underlying CLI does not return a per-call USD figure. Token counts may still be recorded where the CLI emits them, but the cost column will be blank. If a task mixes CLI seats and OpenRouter seats, the aggregate at the bottom reflects only the OpenRouter portion; the CLI portion is already accounted for inside your subscription. Exports include the same per-run table, so a saved decision record carries the cost breakdown forward.

11.2 Tier-based retention

Retention runs in the background on a fixed cadence — every `21600` seconds (6 hours) — via the worker started at service launch. Each pass is a no-op unless an operational trigger fires: either the on-disk database (SQLite file plus its WAL and SHM sidecars) exceeds `max_db_size_mb` (default `2048` MB, i.e. 2 GB), or the count of terminal tasks (`completed`, `failed`, `cancelled`) exceeds `max_completed_tasks` (default `1000`). Either condition alone is enough to trigger a pass.

Selection is tier-based, not time-based. Tier 1 — decisions, charter amendments, and unresolved dissent — is never auto-trimmed. Tier 2 — the `final_results` row carrying the conclave's verdict and agreement level — is retained until you explicitly export it. Tier 3 — the `agent_messages` rows holding the per-turn transcript — is what gets trimmed first, oldest task first, only for tasks that are terminal, summarized, unreferenced by any child thread, and either reached consensus or have a user decision recorded.

An absolute minimum age applies on top: tasks newer than `min_task_age_days` (default `90`) are never trimmed regardless of triggers or tier. After a trim, `VACUUM` runs so SQLite actually returns the space to disk.

Opt-in Tier 2 trim after export. Setting `retention.trim_tier2_after_export: true` in `config.yaml` lets the worker also drop `final_results` rows for tasks whose markdown export already exists on disk (the export is the long-term archive). Tier 2 only kicks in if Tier 3 trimming was insufficient to bring the database back under budget. The flag is off by default — turn it on once you trust your export workflow.

11.3 Tier 2 export tracking

Each task has two columns supporting export tracking: `exported_at` (timestamp of last export) and `export_path` (path of the written file). Both are `NULL` for tasks you have never exported.

Exporting a task — either via the "Export to decision record" button in the Detail view or by hitting `POST /api/tasks/{id}/export` directly — writes a markdown snapshot to `<runtime-state-root>/exports/<task_id>.md` and stamps both columns. The Inbox shows an "exported" indicator on tasks that have been exported, and the label on subsequent exports of the same task switches to "Re-export" so you know you are overwriting an existing snapshot rather than producing the first one.

For bulk export, `POST /api/tasks/export-batch` iterates every terminal task with a `NULL` `exported_at` and writes each one to the same exports directory. This is the "snapshot everything I have not preserved yet" path — useful periodically, particularly before any retention work you want to be sure does not touch unsaved verdicts. The Inbox's export-status filter lets you scan for what is still unexported before running it.

The export columns are also what the (now shipped, opt-in) `retention.trim_tier2_after_export` flag hooks into — when set, the retention worker considers exported tasks eligible to drop their `final_results` rows once Tier 3 trimming alone is not enough to come back under budget.

11.4 What is never auto-trimmed

Three categories are immune to auto-trimming. First, Tier 1 content: the task row itself (which carries `user_decision` and the `parent_task_id` link that keeps threads walkable), charter amendments, and any task with unresolved dissent — i.e. `agreement_level` in `major_disagreement` or `unresolved` with no `user_decision` recorded. Those are treated as open questions awaiting your judgment and are never touched.

Second, anything newer than `min_task_age_days` (default `90` days). This is a hard floor independent of tier — a recent task with cleanly converged consensus and no thread children still will not be trimmed if it is younger than the threshold.

Third, anything still referenced as a parent by a live thread. As long as another task points at it via `parent_task_id`, the orchestrator keeps the messages around so `/thread` and `/continue` can walk the ancestry.

You can tune all of this in `config.yaml` under the `retention` section. Set `enabled: false` to disable retention entirely; raise `max_db_size_mb` or `max_completed_tasks` to push the triggers out; lengthen `min_task_age_days` if you want a longer untouched window. The retention pass logs a summary line on every actual trim, so you can confirm exactly which task IDs were touched.

11.5 Confidence-weighted convergence

Every conclave participant emits a `confidence` score (0.0–1.0) on every turn. The system collects these turn-by-turn already, but Phase 2 of the post-DR plan on `tsk_01KRSW6AS3M66B4RRJE3JFAPRV` made them visible: the Final Result panel now shows an aggregate stat row (min / mean / max / participant count) with green / amber / red banding, and below it a round-by-round dot trail per participant. That round trail is the "who-persuaded-whom" record — you can see who held firm (codex 0.86 → 0.95 in a five-round task), who reached certainty fast (gemini 0.85 → 1.00), and who actually reconsidered mid-deliberation before joining the consensus (claude-code dipping to 0.65 in round 3, climbing back to 0.97 by round 5).

When the spread between max and min is at least `0.30` across two or more participants, a caveat fires explicitly: "Wide spread ... consensus may be conformist drift rather than robust agreement." This is exactly the failure mode the Charter v1.1 Multimodal Disagreement clause was designed to flag — a "consensus" reading of `agreement_level=consensus` that's actually $1 \times 0.95 + 3 \times 0.4$ instead of 4×0.95 . The synthesis directive (the one-shot round after weak convergence) also now includes each participant's confidence inline so the synthesizer can weight positions by stated certainty rather than treating all voices as equal.

The aggregate is persisted in `final_results.confidence_aggregate_json`; the trajectory is computed on-the-fly from `agent_messages` at view time (works for any historical task, not just post-Phase-2 ones).

11.6 Decision Memory (Prior Art)

Every new task automatically retrieves the most relevant past decision records and surfaces them as a *Prior Art* section, both in the prompts the agents see and as a gold-accented panel in the Detail view between the user request and the transcript. The retrieval is TF-IDF cosine similarity over `docs/decisions/*.md` — zero external dependencies, computed at task-creation time, and frozen so the user sees later exactly what the agents saw when they deliberated.

The top three matches with a relevance score ≥ 0.05 are persisted in `tasks.prior_art_json`. The injected prompt section reads, for each match, the decision number ("Decision 11"), title, date, relevance score, the summary paragraph from the record's *What Was Chosen* section, and the file path under `docs/decisions/`. Agents are explicitly told to consult them and either cite a record that settles the question or say why it doesn't apply — the goal is to operationalize the Charter's "decisions are reusable context for future work" mandate so settled questions don't get re-litigated.

Real-corpus behavior: "OpenRouter API key storage" hits Decision 11 (relevance `0.13`) plus Decision 10 (relevance `0.07`); "multimodal disagreement" hits Decision 2 (`0.26`); "export task as pdf" hits Decision 8 (`0.21`); unrelated queries return nothing. The index is rebuilt automatically when any file under `docs/decisions/` changes.

11.7 Orphan task reaper

A small startup sweep handles the worker-crash failure mode where a task ends up stuck in `running` forever. The reaper (`app/services/orphan_reaper.py`) runs once at service launch, scans for any task whose status is still `running` and whose `updated_at` is older than one hour, marks it `failed` with the message "orphaned: no progress for >1h; marked failed by startup reaper", and writes a `task_orphaned` entry to the `logs` table.

The transcript and any agent_runs / final_results rows are preserved untouched — only the task status changes. Idempotent (re-running on the same DB does nothing once everything is marked failed), wrapped in a try/except so a reaper failure never blocks service startup, and tunable via the `threshold_hours` parameter if you ever want a tighter or looser window. Zero UI surface by design — the full user-facing recovery console (inspect last durable state, requeue, export recovery bundle) is intentionally deferred until stuck tasks turn out to be a recurring problem.

11.8 OpenRouter tool-loop (per-seat opt-in)

All four OpenRouter-backed seats (`deepseek`, `glm`, `qwen`, `kimi`) support **tool-loop mode**, configured per seat in `config.yaml`. As of 2026-05-18 the default ships `tool_loop: true` on every seat — the DR0015 v1 "verify each seat one at a time before flipping the rest" phase is complete; all four have been observed emitting clean tool_calls on real tasks. When the flag is on AND a task has a sandbox, the adapter exposes three function-calls to the model — `read_file(path)`, `list_dir(path)`, `glob(pattern)` — and loops POST → tool_call → tool_result → POST until the model emits its structured turn JSON or a per-turn cap fires. The agent reads exactly what it actually needs instead of receiving the entire project sandbox inlined at task creation. Ratified as DR0015.

Why this matters. Without tool-loop, on a code-heavy conclave with ~95K-token inlined prompts, the open-weight seats fall back to template best-practices because their effective attention budget (somewhere in the 50–80K range) is exhausted before they get to your actual code. With tool-loop, the same model on the same task pulls the 5–10 files it actually needs and reasons over those — strictly better signal per token.

How to toggle. In `config.yaml`, under `openrouter.models[]`, set `tool_loop: true` (or `false` to revert) on the specific seat. Submit a task with that seat included and watch the transcript: each tool call and its result appears as a compact ribbon entry under the agent's row, click-to-expand for the full payload. If the model behaves cleanly (no infinite loops, no malformed calls, the structured turn JSON looks right), keep it on. If something looks off, flip it back to `false` — no code change, just a restart.

Bounds. Per-turn: maximum 8 tool calls, maximum 256 KiB cumulative tool-response bytes, and maximum 3 consecutive malformed tool calls. Any one of these fires a forced final-turn POST instructing the model to emit its structured turn now with whatever it has. The forced-final case is logged so you can spot a seat that's chronically running into a cap (and tune the bounds in code, or disable the seat). Per-call: `read_file` caps content at 80,000 chars with a truncation signal; `glob` caps results at 200 paths.

Safety. Read-only access to the same sandbox the seat already had inlined under DR0012 — no capability expansion. Path-traversal attempts (`../`, absolute paths escaping the sandbox, symlinks pointing out) are refused with an error returned to the model so it can self-correct. The same ignore set the sandbox already applies (`.git`, `node_modules`, `__pycache__`, build outputs, binaries, ...) is honored by all three tools.

Audit trail. Every `tool_call` and `tool_result` is persisted as its own row in `agent_messages`, linked to the same `agent_run` as the structured turn that followed. Exports (markdown / PDF / DOCX / TXT) include them. You can post-hoc reconstruct exactly which files an agent read, the order it read them in, how big each result was, and whether any call failed.

Fallback. When `tool_loop: false` (set per seat to opt out) the seat uses the DR0012 inlined-sandbox path unchanged. If a seat is configured `tool_loop: true` but the task has no sandbox attached, the adapter quietly falls back to the prompt-only path for that task. If a model is configured for tool-loop but doesn't implement function-calls cleanly, the loop will likely produce content immediately without ever emitting `tool_calls` — which is still a valid (if degraded) outcome; the adapter accepts it as the final turn.

12. Exports

12.1 Per-task detail export (PDF / DOCX / MD / TXT)

Per-task detail export lets you save a single deliberation in your format of choice: `PDF`, `DOCX`, `MD`, or `TXT`. The control lives in the Detail view's post-task bar — an "Export detail as..." selector next to a Download button. Pick the format, click Download, and your browser's Save dialog (or `showSaveFilePicker` on browsers that support it) decides where the file lands.

Under the hood, the dashboard hits `GET /api/tasks/{id}/download?format=...` and streams the bytes back. Each format renders the same section order so the four outputs are recognizable as the same document: header (task id, mode, status, participants, source agent, timestamps), the original question, the recorded decision (if any), the final result (including agreement level and any disagreements), and the full transcript verbatim — every agent turn, every structured field, no truncation of prose. The PDF and DOCX paths live in `app/services/doc_export.py` (ReportLab and `python-docx` respectively); the markdown path lives in `app/services/exporter.py`; the plain-text path is a straightforward strip of the markdown output.

This is a snapshot operation — it does not touch the database or mark the task as exported. It is the right path when you want a portable, shareable copy of the deliberation; it is not the path for the Tier 2 "I want this preserved" semantics. For that, use the decision record export described next.

12.2 Decision record export

Decision record export is the canonical "I want to keep this" action for a terminal task. The "Export to decision record" button on the Detail view (or `POST /api/tasks/{id}/export` from the API) writes a markdown snapshot to `<runtime-state-root>/exports/<task_id>.md` and stamps the task's `exported_at` and `export_path` columns so the system knows you have preserved it.

The contents mirror the markdown export pipeline in `app/services/exporter.py`: a header with task id, mode, task type, agents, timestamps, and any parent task; the original question fenced verbatim; the recorded decision (with the timestamp you recorded it at); the final result, including `agreement_level`, `resolution_status`, any disagreements with primary and consultant positions, recommended actions, risks, and commands or patches that would require approval; the full transcript grouped by round for conclave mode (flat for the others) with every agent's structured fields preserved; and a usage table showing per-run tokens and USD cost plus the task totals.

Two things to note. It overwrites in place — re-exporting the same task replaces the prior file at `<runtime-state-root>/exports/<task_id>.md` and the dashboard label flips to "Re-export". And it is the action the retention system watches: when `retention.trim_tier2_after_export` is on (opt-in), a task with `exported_at` set becomes eligible to drop its `final_results` row during a later retention pass.

12.3 Bulk export

Bulk export is the inverse of one-at-a-time decision record export. From the Inbox, the "Bulk export unexported" action iterates every terminal task with a `NULL` `exported_at` and writes each one to `<runtime-state-root>/exports/<task_id>.md`, stamping `exported_at` and `export_path` as it goes. The underlying endpoint is `POST /api/tasks/export-batch`.

The output is identical to the per-task decision record export — same renderer in `app/services/exporter.py`, same header / question / decision / final result / transcript / usage layout, same overwrite semantics if a file is somehow already present.

Useful patterns. Run it periodically — say, once a week or before any kind of retention work — to snapshot fresh decisions you have not gotten around to exporting individually. The Inbox's export-status filter lets you preview what is still unexported before running it; sort by date to confirm nothing important is about to roll out from under you. After a run, the Inbox indicators flip to "exported" for every task that was processed.

Bulk export only processes terminal tasks (`completed`, `failed`, `cancelled`) — in-flight tasks are skipped. Tasks that already have an `exported_at` are also skipped, so the operation is idempotent: running it twice in a row does no extra work the second time.

13. Troubleshooting

13.1 Agent unavailable

If an agent shows up as unreachable, the dashboard's New Task form will paint a red dot next to its checkbox with a remediation hint as the tooltip. The same data is available programmatically at `GET /api/health` as a `seats: [...]` array — each entry includes `{available, reason, hint}`. See 6.6 *Seat readiness signals* for the full reason-code table.

For a deeper probe, hit the per-agent test endpoint `POST /api/agents/<name>/test` directly — it actually invokes the CLI's `--version` path rather than just checking presence:

```
curl.exe -X POST http://127.0.0.1:8787/api/agents/codex/test
curl.exe -X POST http://127.0.0.1:8787/api/agents/gemini/test
curl.exe -X POST http://127.0.0.1:8787/api/agents/claude-code/test
```

A healthy agent returns `{"available": true, "version": "...", ...}`. `{"available": false, ...}` means the adapter could not invoke the underlying CLI.

First thing to check: is the CLI actually on `PATH`? On Windows: `where.exe codex`, `where.exe gemini`, `where.exe claude`. On Unix: `which codex`, `which gemini`, `which claude`.

If `where.exe` / `which` finds nothing, install the CLI (see Prerequisites in `INSTALL.md`) or set `agents.<name>.command_path` in `config.yaml` to the absolute path of the binary. Strict precedence applies — if `command_path` is set but the file doesn't exist, the seat reports `configured_path_missing` rather than silently falling back to `PATH`. (This is the usual fix when the CLI works in your terminal but not under a GUI launch — Windows and macOS GUI processes don't inherit shell `PATH`.)

If the CLI is on `PATH` but the adapter still reports unavailable, the most common cause is authentication. Run the CLI directly with `--version`. Confirm it does not prompt for login. If it does, re-authenticate: `codex login`, `gemini /auth`, or `claude /login`. The Switchboard adapters use your OAuth session by default; they do not pass `--bare` or require `ANTHROPIC_API_KEY`.

If `GET /api/agents` returns `{"agents": []}`, none of your real CLIs are reachable — install at least one or use the `fake` adapter via the API directly.

13.2 JSON parse failures

If an agent call fails with `agent_error: could not extract JSON`, the adapter received output from the CLI but could not parse the structured turn out of it. Two causes account for almost every case.

Most often: the CLI updated and changed its output format. The adapters depend on specific shapes — `codex exec --json`, `gemini -p -o json`, `claude -p --output-format json` — and a CLI version bump can move fields, add envelope wrappers, or switch from JSON to NDJSON. Open the adapter file for the affected agent (`app/agents/codex_adapter.py`, `app/agents/gemini_adapter.py`, `app/agents/claude_adapter.py`) and check that the parsing logic still matches what the CLI emits. The quickest reproduction is to run the CLI directly with the same flags the adapter uses and inspect the raw output.

Less commonly: the prompt was too long and the model gave up before producing structured output. Symptoms here look the same — the adapter received text (an apology, a refusal, a partial answer) but no parseable JSON. Mitigation is to shrink the input: scope `project_path` to a subdirectory rather than the full repo, shorten the question, or drop unneeded attachments. For OpenRouter / vLLM seats, see also the context-overflow section below — those failures have a separate code path.

If the failure is intermittent, capture the raw CLI output (the adapter logs it) and diff successful runs against failing ones to narrow down the trigger.

13.3 Stuck tasks

If a task sits in `running` indefinitely — long past any reasonable per-agent timeout — the worker has almost certainly crashed mid-task. The task's status field is updated by the worker that owns it, so if the worker dies, the field stays stuck.

The fix is to restart the service. Two sweeps run at startup. First, the orphan-sandbox sweep cleans up any per-task sandbox directories under the runtime `sandboxes/` root whose owning task is no longer live. Second, the orphan-task reaper (`app/services/orphan_reaper.py`) scans for any task still in `running` whose `updated_at` is older than one hour, marks it `failed` with the message `"orphaned: no progress for >1h; marked failed by startup reaper"`, and writes a `task_orphaned` entry to the `logs` table. The transcript, `agent_runs`, and any `final_results` rows are preserved verbatim — only the task status changes.

So in practice: bounce the service and the stuck task moves itself to `failed`. If for some reason you need to intervene faster than that — e.g. you want to clear a task mid-session — the manual SQL path still works (substitute your resolved runtime-state root for `<state-root>`):

```
sqlite3 <state-root>/switchboard.db "UPDATE tasks SET status='failed', error_message='worker cr"
```

The full user-facing recovery console (inspect last durable state, requeue from last safe point, export recovery bundle) is intentionally not built — it would only earn its keep if stuck tasks become a recurring problem in practice rather than a once-in-a-blue-moon occurrence.

13.4 Context-overflow errors

The OpenRouter and vLLM-style seats run into context limits in a way the CLI seats usually do not: they receive a single inlined prompt rather than browsing files on demand, so a large project sandbox or a thick stack of attachments can push past the model's effective window.

The adapters mitigate this in two ways. First, each adapter self-discovers the model's context limit from the provider when it can, instead of trusting `max_context_chars` blindly. Second, on a context-overflow response, the adapter automatically retries with a tighter budget — trimming the inlined sandbox or attachments down before the next attempt.

If retries still fail, you will see an actionable error on the task. The recommended fixes, in order of effort: scope `project_path` to a subdirectory rather than the repo root so less code is inlined; remove non-essential attachments from the request; or raise the seat's `max_context_chars` in `config.yaml` under the `openrouter.models` entry if your model actually has a larger window than the conservative default. The defaults in `config.example.yaml` are set well below each model's true token window because code-heavy prompts hit roughly 3 chars/token, worse than prose's 4.

The structural fix — a tool-loop architecture letting the API seats `read_file` / `list_dir` / `glob` on demand instead of getting everything inlined — has now shipped as DR0015 and is opt-in per seat (see *11.8 OpenRouter tool-loop*). Flipping `tool_loop: true` for a seat that keeps tripping over its context window is usually a better fix than narrowing scope.

13.5 Sandbox cap exceeded

The per-task project sandbox has a hard cap of `200 MiB`. The sandbox is built by copying `project_path` after standard ignore patterns are applied (the same patterns the drag-a-folder upload uses, plus VCS metadata, build outputs, and similar). If your project still exceeds 200 MiB after that, the copy stops at the cap, the service logs a `cap reached` line in the service logs, and the sandbox the agents see is truncated.

You will notice this two ways: the log line on the service side, and — at deliberation time — agents asking about files that obviously should exist or basing analysis on a partial view of the codebase. The truncation is a hard stop, not a smart selection: whatever ran past 200 MiB was simply not copied.

The fix is to narrow scope. Point `project_path` at the subdirectory you actually need the conclave to look at (e.g. `app/` rather than the whole repo root, or the specific package being reviewed) and re-submit. That is almost always the right move anyway — a tighter scope produces a better signal-to-noise ratio in the deliberation than dumping the entire monorepo at the agents.

If you genuinely need a larger sandbox, raising the cap is a code change in `app/services` rather than a config knob; the 200 MiB ceiling was set deliberately to keep per-task disk usage bounded and the orphan-sweep cleanup tractable.

14. Reference

14.1 Glossary

Term	Definition
<code>Conclave</code>	N equal AI participants deliberating with full-mesh visibility — no primary, no winner picked by the orchestrator.
<code>Council</code>	Generic term for the set of agents on a task, regardless of mode.
<code>Primary</code>	In <code>resolve</code> and <code>consult</code> modes, the agent driving the loop.
<code>Consultant</code>	In <code>consult</code> mode, an agent providing critique on the primary's proposal.
<code>Judge</code>	After weak conclave convergence and a synthesis round, one participant arbitrates semantic equivalence so <code>agreement_level</code> can be upgraded.
<code>Keeper</code>	You. Records <code>/decide</code> (ratify / reject / return-to-conclave); does not edit conclave output.
<code>Charter</code>	The binding agreement (currently v1.2) embedded into every agent prompt by the prompt builder.
<code>Decision record</code>	Numbered Markdown file in <code>docs/decisions/</code> capturing what was chosen, why, what was rejected, known risks, and open questions.
<code>Sandbox</code>	Per-task read-only copy of <code>project_path</code> that agents can browse during a deliberation.
<code>Tier (1/2/3)</code>	Retention tiers. Tier 1 = never trimmed (decisions, charter amendments, unresolved dissent); Tier 2 = retain until exported; Tier 3 = agent messages, trimmed first.
<code>Convergence</code>	In <code>conclave</code> mode, the threshold (default <code>1.0</code> = unanimous) of participants who must signal <code>i_am_done</code> for the deliberation to terminate.
<code>Source agent / provenance</code>	The <code>source_agent</code> column recording which surface submitted the task: <code>claude-code</code> , <code>codex</code> , <code>gemini</code> , <code>dashboard</code> , or <code>api</code> .
<code>Operability</code>	From Charter v1.2: every capability must preserve observability, durability, recoverability, audit trail, retention, and export.

Term	Definition
Thread	Chain of related tasks linked via <code>parent_task_id</code> ; surfaced through <code>/continue</code> and <code>/thread</code> .
Mode	Shape of the deliberation: <code>conclave</code> , <code>resolve</code> , <code>consult</code> , <code>handoff</code> , or <code>poll</code> .
Agreement level	Final-result classification of how aligned the agents ended up: <code>consensus</code> , <code>minor_disagreement</code> , <code>major_disagreement</code> , or <code>unresolved</code> .
Adapter	The per-tool subclass of <code>BaseAdapter</code> that encapsulates one agent's CLI or HTTP quirks behind the orchestrator's uniform interface.

14.2 Keyboard shortcuts

Shortcut	What it does
?	Open Help. Ignored while you are typing in an <code><input></code> , <code><textarea></code> , <code><select></code> , or any <code>contenteditable</code> element.
Enter / Space	On the file dropzone (when focused): open the native file picker.
Esc	No global <code>Esc</code> handler is wired yet.

Those are the only global shortcuts at the moment. The list will grow as the dashboard does.

14.3 File locations

Runtime-state paths below are relative to the resolved *runtime state root* — see 14.4 *Where state lives* for how the root is chosen. The "repo" paths assume you're running from a Switchboard checkout.

Path	What it holds
<code><state-root>/switchboard.db</code>	SQLite database. Every task, message, decision, and transcript lives here (plus the <code>-wal</code> and <code>-shm</code> sidecar files).
<code><state-root>/sandboxes/<task_id>/</code>	Per-task read-only sandbox — the copy of <code>project_path</code> that agents may browse during the deliberation.
<code><state-root>/uploads/</code>	Task attachments uploaded from the dashboard or the API.
<code><state-root>/exports/</code>	Exported decision records and per-task downloads (PDF / DOCX / Markdown / text).
<code><state-root>/logs/</code>	Reserved for durable file logging. Today's runtime logs are stderr-only; the directory exists so a future record can adopt it without further plumbing.
<code><state-root>/switchboard.pid</code>	Single-instance pidlock. Delete only if you're certain no service is running.
<code><repo>/config.yaml</code>	Operator configuration. Copy from <code>config.example.yaml</code> and edit ports, adapter paths, <code>command_path</code> overrides, and the open-weight seat list. In a packaged install, the resolver also checks <code><state-root>/config.yaml</code> first.
<code><repo>/pyproject.toml</code>	Project metadata. Also serves as the dev-mode anchor — its presence (alongside <code>config.example.yaml</code>) is how <code>user_data_root()</code> decides the runtime state root resolves to <code><repo>/data/</code> .

Path	What it holds
<code><repo>/launch.pyw</code>	Cross-platform desktop launcher (see 6.5 <i>One-click desktop launcher</i>). The installer scripts in <code>tools/</code> wire shortcuts to it.
<code><repo>/data/launcher.log</code>	Launcher events plus uvicorn stdout/stderr — the first place to look if the desktop launcher fails to open a browser.
<code>~/.claude/commands/</code>	Claude Code slash commands, installed by <code>python clients/install.py</code> .
<code>~/.claude/skills/switchboard-conclave/</code>	Claude Code natural-language skill that backs the conclave triggers.
<code>~/.codex/skills/switchboard-conclave/</code>	Codex skill — Codex has no literal slash commands, so it activates from trigger phrases.
<code>~/.gemini/extensions/</code>	Gemini extension link installed by the clients installer.

Everything under the runtime state root is gitignored (in dev mode) and never leaves your machine. The database contains the full text of every deliberation plus any source copied into per-task sandboxes — treat it as sensitive.

14.4 Where state lives

Per DR0016, all writable runtime state — the SQLite database, sandboxes, uploads, exports, logs, and pidlock — resolves through a single `user_data_root()` primitive in `app/utils/paths.py`. The root is picked at startup using this precedence (first match wins):

- 1. `SWITCHBOARD_DATA_DIR` environment variable. Wins over everything else. Tests, CI runs, and packaged installers use this hook.
- 2. Dev-mode anchor. If the current working directory (or any ancestor) contains both `pyproject.toml` and `config.example.yaml`, the root resolves to `<that-dir>/data/`. This preserves the in-repo developer experience exactly — running `python -m uvicorn ...` from a checkout still writes to `./data/`.
- 3. Platform-conventional user-data directory. Used by packaged installs that don't ship inside a repo checkout.

Launch mode	Resolved root
Dev / running from the repo	<code><repo>/data/</code>
Packaged build on Windows	<code>%LOCALAPPDATA%\AI Switchboard\</code>
Packaged build on macOS	<code>~/Library/Application Support/AI Switchboard/</code>
Packaged build on Linux	<code>\$XDG_DATA_HOME/ai-switchboard/</code> (or <code>~/.local/share/ai-switchboard/</code>)
Explicit override	Whatever <code>SWITCHBOARD_DATA_DIR</code> points at.

Config discovery has its own precedence, layered on top: `SWITCHBOARD_CONFIG` env var, then `<state-root>/config.yaml`, then dev-mode `./config.yaml`, then `./config.example.yaml`, then built-in defaults. The example file is checked in; the live `config.yaml` is gitignored.

First-run migration. When a packaged build launches and finds a populated `./data/` from a prior dev-mode run, it copies (not moves) the state into the new root before opening any writer: the DB is transferred with `VACUUM INTO` for a consistent single-file snapshot, then `sandboxes/`, `exports/`, and `uploads/` are copied via atomic `.tmp` rename. The original files at `./data/` are left intact so you can verify the migration before deleting them. If a live process holds `./data/switchboard.pid`, migration refuses to run and tells you to stop the old service first.

14.5 Links to deep docs

- `docs/CONCLAVE_CHARTER.md` — The binding charter (v1.2), embedded in every agent prompt.
- `docs/SWITCHBOARD_PROTOCOL.md` — Wire format, mode definitions, and message schemas.

- `docs/AGENT_ADAPTERS.md` — The interface every adapter must satisfy.
- `docs/TASK_LIFECYCLE.md` — The task state machine, including the pause / resume paths.
- `docs/CODING_WORKFLOW.md` — The canonical four-step loop: deliberate, decide, execute, record.
- `docs/SAFETY_MODEL.md` — Permission model and approval rules.
- `docs/ROADMAP.md` — Shipped, next, and intentionally-not-built.
- `docs/decisions/INDEX.md` — Every ratified decision with a one-line summary.
- `docs/DATABASE_SCHEMA.md` — Table reference for `data/switchboard.db`.
- `docs/SKILLS_SYSTEM.md` — How skill files compose into the prompt sent to each agent.
- `clients/README.md` — Slash-command parity across Claude Code, Codex, and Gemini.
- `CLAUDE.md` — Guidance for Claude Code instances working in this repo.

15. About This Documentation

15.1 Version history

This document carries its own version number, bumped whenever the docs are edited. The scheme is semver: a **patch** bump for typos and small fixes, a **minor** bump when new sections or substantive content land, and a **major** bump if the structure is reorganized.

The current version and date are shown in the header at the top of this page. There is no separate changelog yet — for now, the *Updated* date in the header is the signal that something has changed.

Print / Save as PDF. The button in the help titlebar opens your browser's print dialog. Choose *Save as PDF* as the destination to capture a self-contained copy of this document — `@media print` CSS hides the chrome (toolbar, splitter, version chip) and forces a white background so the printed output is the prose only. Useful for offline reference or sharing the doc as a snapshot of what shipped at a particular version.

15.2 How currency is detected

When you open Help, the dashboard calls `/api/help/metadata`, which returns `{doc_version, covered_app_version, last_updated, app_version, currency}`. `covered_app_version` is the app version this document was last audited against; `app_version` is what is actually running right now. The two are compared on **major.minor only** — patch differences stay green.

The badge in the header reflects one of four states:

Current — major.minor match. The documentation is aligned with the running app.

App is newer — your app has newer features than the doc covers. Some recent additions may not be documented yet.

Older app — you are running an older app than the doc was written for. No warning is shown.

Unknown — one or both version strings did not parse.